

Does Removing Non-Anatomical Artifacts from Chest X-Rays Improve Pneumonia Classification?

Aleksandr Sorokin

Viggo Beck
Hero Englund

Sebastian Drumm

Applied Artificial Intelligence — Exam Project

June 5, 2026

Abstract

Chest X-rays often contain small letters and markings that are not part of the body, such as an *R* or *L* that shows which side is which. A neural network might use these markings as a shortcut instead of actually learning to recognise pneumonia. In this project we test whether removing them makes a pneumonia classifier better. We used the Kaggle *Chest X-Ray Images (Pneumonia)* dataset (5,856 images), split it into training, validation and test sets (80/10/10), and trained a convolutional neural network (CNN) following Nunamaker et al.’s research process. We then ran a controlled experiment: we trained one model on the original images and one on a cleaned copy with the markings removed, keeping everything else exactly the same. The cleaned dataset did not improve test performance: it raised specificity (+0.019) but lowered sensitivity by more (−0.030), giving slightly lower Youden’s *J* (−0.0115), accuracy (−0.0170) and AUC (−0.0013) overall. We conclude that, for this dataset and model, removing the markings does not give a measurable benefit: the model’s decisions are driven by the lungs rather than the markings.

1 Introduction

Pneumonia is a common and serious lung infection, and the chest X-ray is one of the most common ways to look for it. In recent years, deep learning models have become very good at reading chest X-rays, in some cases matching doctors [1]. This makes them useful as a tool to help sort patients.

A known problem with these models is *shortcut learning*. This is when a model gets good results by using the wrong information — something that happens to be linked to the answer but is not the real cause [2]. Chest X-rays are a good example. Earlier studies found that models sometimes used hospital-specific text printed on the image [3], or that COVID-19 X-ray models often looked at markers and image edges instead of the lungs [4]. Our dataset has the same kind of markings: most images have a bright *R* or *L* marker, and some have extra text near the border (Figure 3).

This leads to a clear question we can test: if we remove these markings, does the pneumonia classifier get better? If it does, the model was partly relying on them. If it does not, that is a sign the markings are not what the model uses. We test this with a controlled experiment.

2 Research Question

We phrase one main question with two supporting sub-questions.

Q1 Does removing non-anatomical artifacts (e.g. the orientation marker *R*) from chest X-rays improve pneumonia classification performance?

Q1.1 How can the markings be detected and removed reliably and quickly, without damaging the lungs?

Q1.2 How should the strong imbalance between NORMAL and PNEUMONIA images be handled so it does not affect the comparison?

To answer Q1 we keep *every* other factor the same between two training runs and change only one thing — whether the images still contain the markings — so that any difference in test performance must come from the cleaning alone.

3 Methods

We follow Nunamaker et al.’s multi-methodological research process [5], which moves back and forth between observing the data, building the system and running experiments. Every step from loading the data to the final evaluation is written in code so the results can be reproduced. We also fix the random seed everywhere (Python, NumPy and PyTorch) so the splits, sampling and training give the same result each time.

3.1 Data and splitting

We use the Kaggle *Chest X-Ray Images (Pneumonia)* dataset [6, 7], which contains 5,856 chest X-ray images of children, each labelled NORMAL or PNEUMONIA. The

original dataset has a very small validation set (only 16 images), so we combined all the images and split them again into 80 % training, 10 % validation and 10 % test. We use the same split for both the original and the cleaned dataset, so the two are compared on exactly the same images (Section 3.6).

3.2 Preprocessing

Before training we prepare every image in the same way. We convert it to grayscale, pad it to a square so it is not stretched, and resize it to 224×224 pixels, which is a common input size for these models [8]. We also scale the pixel values to a fixed range. For the training images only, we add small random changes — slight rotation, shifting and zooming. This is called *data augmentation*, and it helps the model generalise instead of memorising the training images [9]. The validation and test images are not changed in this way.

3.3 Class imbalance

The dataset is uneven: about 73 % of the images are PNEUMONIA and only 27 % are NORMAL. If we ignore this, the model can become biased towards the larger class [10, 11]. To avoid this we use a *weighted sampler* during training, which shows the smaller class (NORMAL) more often so each training batch has a more even mix. We do this only for training; the validation and test sets keep their natural mix so the results reflect a realistic situation.

3.4 Model architecture

We use a custom convolutional neural network (CNN) for the image classification task. CNNs are commonly used for image classification because they can learn visual patterns directly from image data [14, 15]. In this project the model consists of three convolutional blocks (Figure 1). Each block contains a convolutional layer, batch normalization, a ReLU activation function, and max-pooling. Batch normalization is used to make the training more stable [12].

The number of channels increases through the network from 32 to 64 and finally to 128. This means the model starts by learning simple visual patterns, such as edges and contrast, and later combines these into more complex image features.

After the convolutional blocks, the feature map is flattened and passed through a fully connected classifier. This classifier contains one hidden layer with 256 units, dropout to reduce overfitting [13], and a final output layer with two classes (NORMAL and PNEUMONIA). In total the model has about 25.8 million parameters.

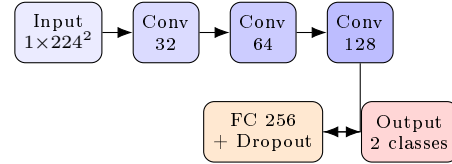


Figure 1: The three-block CNN. Each Conv block is a convolution + batch norm + ReLU + max-pool; the flattened features go into a dropout classifier.

3.5 Detecting and removing the markings

The markings we want to remove (the *R/L* markers and technician text) are small, bright spots near the edge of the image, away from the lungs. We detect them with OpenCV [16] using three simple checks: (i) *location* — the spot is near the left, right or top edge (we ignore the bottom, where the body and film strips appear); (ii) *shape* — it is about the size and shape of a letter; and (iii) *surroundings* — it sits on a dark background, which separates a real marker from bright body parts such as ribs. Once a marking is found, we fill in that area using the pixels around it (a method called *inpainting* [17]) so no sharp edge is left behind. This is done by a script (`create_cleaned_dataset.py`) that saves a full cleaned copy of the dataset with the same folder structure.

3.6 The controlled two-dataset experiment

To answer Q1 we train two separate models: one on the original images and one on the cleaned images. Everything else is kept exactly the same — the model, the settings, which images are in each split, the preprocessing, and how we choose the decision threshold. The only difference is whether the images still contain the markings, so any difference in the results can only come from the cleaning.

3.7 Hyperparameter search

Before the main comparison we try to improve the baseline model by testing different training settings. We use a *random search* [18]: rather than trying every possible combination, we randomly pick 20 settings for four parameters — the learning rate, the dropout rate, the weight decay and the batch size — train a model with each, and keep the setting that scores best on the validation set. The first trial uses our default settings, so the search also tells us whether tuning actually found anything better than what we started with.

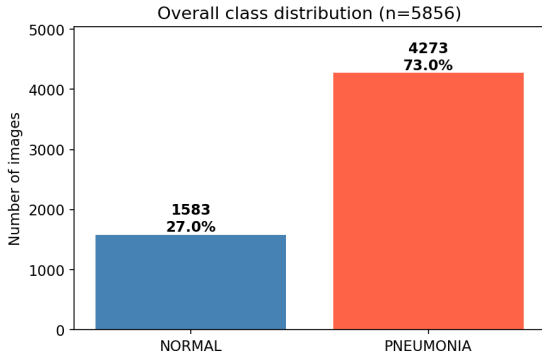


Figure 2: Class distribution across the merged dataset. There are roughly 2.7 times as many PNEUMONIA images as NORMAL images.

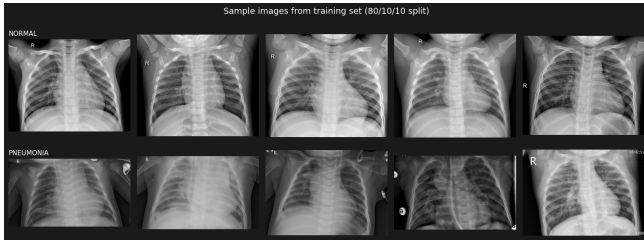


Figure 3: Sample training images (top: NORMAL, bottom: PNEUMONIA). Bright orientation markers can be seen near the borders.

3.8 How we measure performance

Accuracy on its own is not enough here, because missing a real pneumonia case is much worse than a false alarm. So we also report *sensitivity* (how many pneumonia cases the model catches), *specificity* (how many normal cases it correctly leaves out), *precision*, and the *AUC*, which measures how well the model separates the two classes overall. We also use Youden’s J, a single number that combines sensitivity and specificity ($J = \text{sensitivity} + \text{specificity} - 1$) [19]. For each model we choose the decision threshold using the validation set, and then test it once on the held-out test set, which we use only for this final step.

4 Analysis

4.1 Looking at the data

After re-splitting, we have 4,684 training images, 585 validation images and 587 test images. The class imbalance is clear (Figure 2), which is why we use the weighted sampler from Section 3.3. Looking at sample images (Figure 3) we can see the orientation markers, as well as the visual difference between healthy and pneumonia lungs.

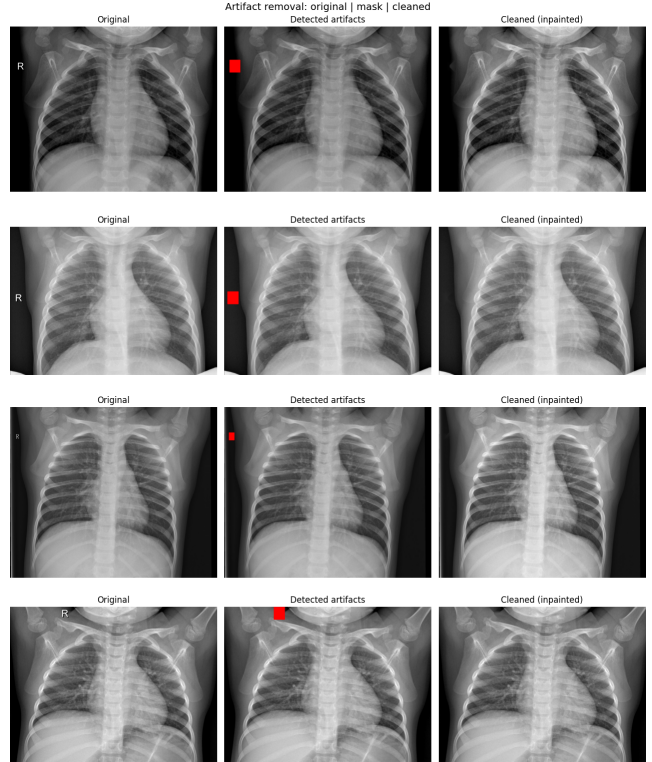


Figure 4: Removing the markings. Left: original. Middle: the detected marker region (red). Right: the image after inpainting. The *R* markers are gone, and the lungs are unchanged.

4.2 Removing the markings

Using the method from Section 3.5, the bright markings near the edges are found and filled in cleanly, while the lungs are left untouched (Figure 4). Checking many images by eye, the three checks almost never trigger on real body parts, which answers sub-question Q1.1.

4.3 Hyperparameter search

The random search (Section 3.7) tested 20 different settings. The best one scored only slightly higher than our default settings on the validation set — an improvement of +0.021 in Youden’s J (Table 1). This tells us the default settings were already close to the best the search could find, so both the default and the tuned model are sensible starting points for the main comparison.

4.4 Original vs. cleaned

We then ran the controlled experiment from Section 3.6. Both models were trained in exactly the same way and tested on the same images. The results — the confusion matrices (Figure 5) and the comparison of metrics (Figure 6, Table 2) — are presented in Section 5.

Table 1: The best random-search trials, ranked by validation Youden’s J. The default settings are included for comparison.

Trial	lr	dropout	wd	Val J	Val AUC
random-3	1.5e-5	0.37	1.2e-5	0.9065	0.9851
random-1	1.9e-4	0.21	6.7e-5	0.9002	0.9870
random-5	2.0e-4	0.42	4.6e-5	0.9002	0.9826
default	1.0e-4	0.40	0.0	0.8852	0.9857

Table 2: Test performance for the two-dataset experiment. Δ is cleaned minus original.

Metric	Original	Cleaned	Δ
Youden’s J	0.8992	0.8877	−0.0115
Sensitivity	0.9369	0.9065	−0.0304
Specificity	0.9623	0.9811	0.0189
Accuracy	0.9438	0.9267	−0.0170
AUC	0.9891	0.9878	−0.0013

5 Findings

On the test set, the model trained on the original images reached a Youden’s J of 0.8992 (sensitivity 0.9369, specificity 0.9623, accuracy 0.9438, AUC 0.9891). The model trained on the cleaned images reached a Youden’s J of 0.8877 (sensitivity 0.9065, specificity 0.9811, accuracy 0.9267, AUC 0.9878). The differences (cleaned minus original) are: −0.0115 in Youden’s J, −0.0304 in sensitivity, +0.0189 in specificity, −0.0170 in accuracy and −0.0013 in AUC (Table 2).

The two changes point in opposite directions. The cleaned model has fewer false positives (6 → 3, so higher specificity), but more false negatives (27 → 40, so lower sensitivity), as shown in the confusion matrices (Figure 5). The drop in sensitivity is the larger of the two, which is why the combined measures (Youden’s J, accuracy and AUC) all end up slightly lower for the cleaned dataset.

6 Conclusion

Coming back to **Q1** (Section 2): for this dataset and this model, removing the markings did *not* improve pneumonia classification. Cleaning produced a trade-off rather than a clear gain. It made the model a little more cautious — fewer normal images were wrongly flagged as pneumonia (higher specificity) — but it also caused more real pneumonia cases to be missed (lower sensitivity). Because missing a real case is the more serious mistake here, this trade-off is unfavourable, and the combined measures (Youden’s J, accuracy and AUC) all came out slightly lower for the cleaned dataset (Table 2, Section 5). So we found no sign that removing the markings helps; the model’s decisions seem to be driven by the

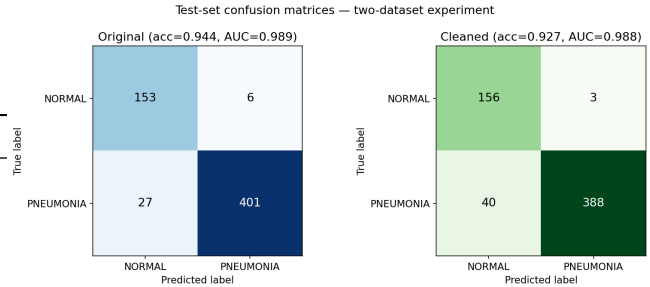


Figure 5: Test-set confusion matrices for the model trained on the original images (left) and on the cleaned images (right).

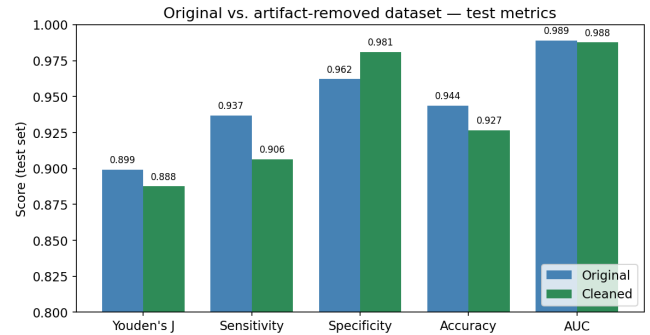


Figure 6: Test metrics for the original and cleaned datasets. The two perform similarly overall; the cleaned model trades higher specificity for lower sensitivity.

lungs, not the printed text. In other words, the short-cut we were worried about [2, 4] does not appear to be helping the model here, probably because the weighted sampler, the augmentation, and the fact that the markers sit outside the lungs already limit their effect.

For the supporting questions: the OpenCV-and-inpainting method (Q1.1) removed the markings reliably and cheaply without harming the lungs (Figure 4), and the weighted sampler (Q1.2) kept the comparison fair by handling the class imbalance.

This study has some limitations. We tested only one CNN, on one split of one dataset of children’s X-rays, and our cleaning only removes bright markings near the edges rather than cutting out the lungs completely, so some non-lung content may remain. A negative result on one split is also not full proof that there is no effect. Future work could (i) repeat the experiment with several random splits and report a range of results, (ii) use a lung-segmentation model (such as U-Net) so the model only sees the lung area, and (iii) use Grad-CAM heatmaps to show directly where the model is looking. That would add visual evidence to support the result we found here.

References

- [1] P. Rajpurkar et al., “CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning,” *arXiv:1711.05225*, 2017.
- [2] R. Geirhos et al., “Shortcut Learning in Deep Neural Networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.
- [3] J. R. Zech et al., “Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study,” *PLOS Medicine*, vol. 15, no. 11, e1002683, 2018.
- [4] A. J. DeGrave, J. D. Janizek, S.-I. Lee, “AI for radiographic COVID-19 detection selects shortcuts over signal,” *Nature Machine Intelligence*, vol. 3, pp. 610–619, 2021.
- [5] J. F. Nunamaker Jr., M. Chen, T. D. M. Purdin, “Systems Development in Information Systems Research,” *Journal of Management Information Systems*, vol. 7, no. 3, pp. 89–106, 1990.
- [6] D. S. Kermany et al., “Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning,” *Cell*, vol. 172, no. 5, pp. 1122–1131, 2018.
- [7] P. Mooney, “Chest X-Ray Images (Pneumonia),” Kaggle dataset, 2018. <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>.
- [8] K. He, X. Zhang, S. Ren, J. Sun, “Deep Residual Learning for Image Recognition,” *CVPR*, 2016.
- [9] C. Shorten, T. M. Khoshgoftaar, “A Survey on Image Data Augmentation for Deep Learning,” *Journal of Big Data*, vol. 6, no. 60, 2019.
- [10] H. He, E. A. Garcia, “Learning from Imbalanced Data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [11] B. Krawczyk, “Learning from imbalanced data: open challenges and future directions,” *Progress in Artificial Intelligence*, vol. 5, pp. 221–232, 2016.
- [12] S. Ioffe, C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” *ICML*, 2015.
- [13] N. Srivastava et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [14] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [15] A. Krizhevsky, I. Sutskever, G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *NeurIPS*, 2012.
- [16] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [17] A. Telea, “An Image Inpainting Technique Based on the Fast Marching Method,” *Journal of Graphics Tools*, vol. 9, no. 1, pp. 23–34, 2004.
- [18] J. Bergstra, Y. Bengio, “Random Search for Hyper-Parameter Optimization,” *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.
- [19] W. J. Youden, “Index for rating diagnostic tests,” *Cancer*, vol. 3, no. 1, pp. 32–35, 1950.